

Lab 08 — Campaign 2: A Social Media Week

Multi AI Agents Workflow for Content Creation (Claude Edition) · Day 2 · Claude Code (agent team) → LinkedIn, Facebook

Course: Multi AI Agents Workflow for Content Creation (Claude Edition) (TGS-2023036153)

Day 2 · Topic 3 · about 50 minutes · slides 88–92

Surface: Claude Code (agent team) → LinkedIn, Facebook

Features: the publisher kit and its hook · an agent team: creator, designer, Lead · teammates message each other · human approval → post

The story so far

Day 2. Week 1 of the campaign starts Monday: two LinkedIn posts and three Facebook posts, each with a visual — and Rachel's one rule: nothing goes out unless a person approved that exact version.

Your goal

Week 1 of the campaign: two LinkedIn and three Facebook posts, each with a visual. Copy and design must agree, the Lead must check every claim — and only a person can approve.

You'll build

content/social/week-01/: five posts with images, reviewed, approved by you and posted (or dry-run)

What is in this folder

- assets/social-brief.md
- assets/publisher-kit/
- assets/publishing-spec.md
- assets/approvals-format.csv
- assets/connect-accounts.md
- assets/env.example
- assets/sample-calendar.csv
- solution/ — reference files from the verified build
- prompts.md / prompts.pdf — every prompt, ready to paste
- evidence/checklist.md — what to capture as proof

Step by step

1. **Install the kit** — Copy publisher-kit/scripts into scripts/ and merge its settings.json into .claude/settings.json (agent teams on, the approval hook). Restart Claude Code.
2. **Test the gate** — Ask Claude to run approve.mjs on anything. The hook must block it.

3. **Start the team** — Paste Prompt A. Teammates appear in the panel; ↑ ↓ and Enter open one; Ctrl+T shows the task list.
4. **Watch them talk** — The designer asks the creator for each post's hook; the Lead sends fixes back.
5. **Approve as a person** — Read each post and image. In your own terminal: `node scripts/approve.mjs <id> --by "Your Name"`.
6. **Publish** — Paste Prompt B. Dry run first; `--live` only if your accounts are connected.

The prompts

PROMPT A — Claude Code: the social team

Create an agent team for Week 1 social from `data/social-brief.md` and `strategy/calendar.csv`.
Spawn two teammates using the agent types `content-creator` and `creative-designer`; you are the Lead.

- `content-creator`: 2 LinkedIn + 3 Facebook posts, one file each in `content/social/week-01/`, with the frontmatter in `data/publishing-spec.md`.
- `creative-designer`: one image per post (Canva or a rendered card) with alt text; message the creator for each post's hook before designing.
- You: fact-check and fin-compliance on every post; message fixes to the teammate who owns it.

When a post passes, run `node scripts/submit.mjs` on it. Publish nothing.

PROMPT B — Claude Code: publish

For every Week 1 post approved in `review/approvals.csv`, run `node scripts/publish.mjs <id>` as a dry run and show me each request. Wait for me to say "go" before any `--live` run.

Check your work

- ☐ The hook (or `approve.mjs`) stopped the agent from approving.
- ☐ Two teammates ran; the task list showed their tasks.
- ☐ The designer and the creator messaged each other.
- ☐ Five posts with images, alt text, frontmatter and UTM links.
- ☐ You approved in your own terminal; the rows show your name and a hash.
- ☐ Dry runs printed every request; nothing went live unless you said go.

If it goes wrong

- **You got subagents, not a team** — Check the env setting in `.claude/settings.json`, restart, and ask for "an agent team".
- **LinkedIn returns 426** — `LINKEDIN_VERSION` must be a recent YYYYMM.

Stretch

- Add the growth-analyst to suggest the best posting time per post from channel-benchmarks.csv.

Why it matters: Subagents report to one boss; teammates also talk to each other. A designer who can ask the writer for the hook makes a better image.

Next

Lab 9 — Campaign 3: Newsletter and a Landing Page. Keep what you built — the next lab starts from it.

Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password, token or client data into a prompt or a file — tokens live only in .env. Every email address in the data must resolve to your own inbox. Nothing is published or sent without a person approving that exact version.